

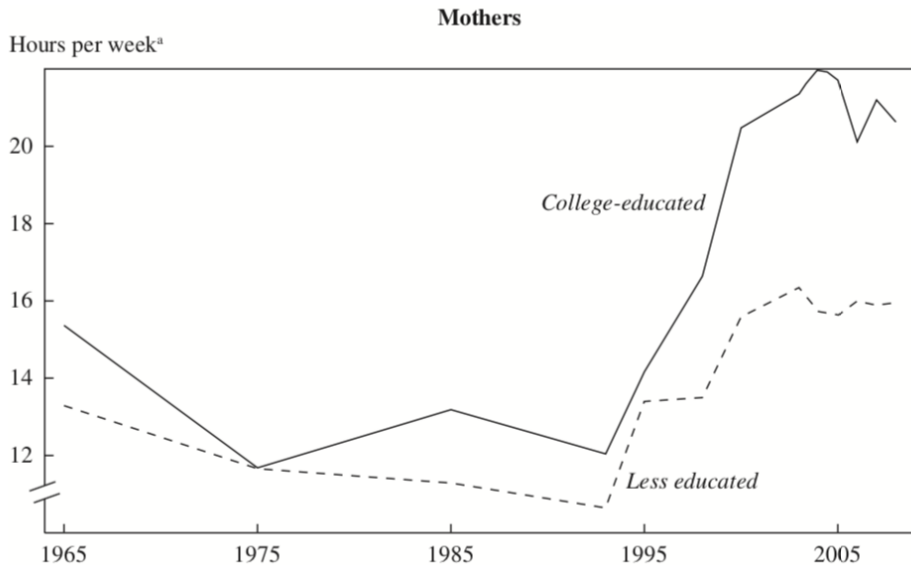
Introduction to Social Data Analytics:

Lecture 7 Part 2

Today's Lecture

- ▶ More functions from the tidyverse package
 - ▶ `mutate()`
 - ▶ `group_by()` and `summarize()`
- ▶ Discuss how to string these new functions together with the pipe operator
- ▶ Revisiting Ramey and Ramey (2010) to study how investment in child care has changed over time and why

Figure 1. Time Spent on Childcare by Parents, by Educational Attainment, 1965–2008



Installing Package

- ▶ If you have never downloaded a package you must either navigate to the “Packages” tab and search for it
- ▶ You can also directly type `install.packages("packagename")` into the R console
- ▶ You only need to do this step once.

Loading a Package

- ▶ In order to use an installed package you need to load it into memory
- ▶ You do this by executing `library(packagename)`
 - ▶ You need to do this in every time you open R
 - ▶ For example, if you load “tidyverse” to complete a quiz this week by executing `library(tidyverse)`, you will have to do this again next time you open R

Installing tidyverse

```
# If it not installed  
#install.packages("tidyverse")
```

```
# load into memory  
library(tidyverse)
```

Read in directly as a tibble

```
rr <- read_csv("rugratrace.csv")
```

```
## Rows: 106020 Columns: 54
```

```
## -- Column specification -----
```

```
## Delimiter: ","
```

```
## dbl (54): state, age, sex, ethnic, under18, under5, ageyngst, student, a
```

```
##
```

```
## i Use `spec()` to retrieve the full column specification for this data.
```

```
## i Specify the column types or set `show_col_types = FALSE` to quiet this
```

Pipe operator

- ▶ Many times in R we want to string along a number of commands
- ▶ For example, in our data, we may want to filter to the relevant observations and then select the relevant variables
- ▶ We can string together these different functions using what is called the pipe operator `%>%`

Pipe operator

- ▶ $x \%>\% f$ is equivalent to $f(x)$
- ▶ $x \%>\% f(y)$ is equivalent to $f(x, y)$
- ▶ $x \%>\% f \%>\% g \%>\% h$ is equivalent to $h(g(f(x)))$

Filtering (with and without pipe operator)

#Without pipe

```
mothers2534 <- filter(rr, age<=34 & age>=25 & mother2==1)
```

#With pipe

```
mothers2534 <- rr %>% filter(age<=34 & age>=25 & mother2==1)
```

Say we wanted to subset, then select

#Without pipe

```
mothers2534 <- filter(rr, age<=34 & age>=25 & mother2==1)  
mothers_select <- select(mothers2534, age, chldtot, college)
```

#With pipe

```
mothers_select <- rr %>%  
  filter(age<=34 & age>=25 & mother2==1) %>%  
  select(age, chldtot, college)
```

Creating new variables

- ▶ We often want to create new columns of our data
- ▶ Many times, these are functions of the other columns

General Syntax of mutate()

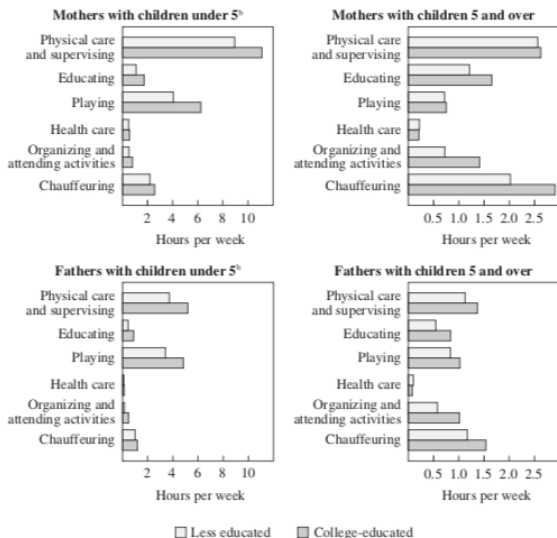
- ▶ The general syntax of mutate() is

```
mutate(newvar = exp)
```

- ▶ Where newvar is the name of a new variable and exp is some expression

Types of Childcare (Ramey and Ramey 2010)

Figure 3. Parental Time Spent on Childcare, by Age of Child and Type of Care, 2003–08^a



Source: Authors' estimates using data from various time-use surveys.

Using data frames, we do this with <-

```
rr <- read.csv("rugratrace.csv")  
#Create travel and education  
rr$childcollegeprep <- rr$childtravel + rr$childeduc
```

► Using tidyverse, we can do this with mutate

```
#Create travel and education  
rr <- mutate(rr,  
  childcollegeprep = childtravel + childeduc)
```

We can create multiple variables

```
#Create travel and education  
rr <- mutate(rr,  
  childcollegeprep = childtravel + childeduc,  
  childnotcollegeprep = childtot - childcollegeprep  
)
```

You can use the variables created earlier in mutate

```
#Create travel and education  
rr <- mutate(rr,  
  childcollegeprep = childtravel + childeduc,  
  childnotcollegeprep = childtot - childcollegeprep  
)
```

Don't forget to overwrite the original data frame

```
#Create college prep and not college prep  
mutate(rr,  
  childcollegeprep = childtravel + childeduc,  
  childnotcollegeprep = childtot - childcollegeprep  
)
```

- ▶ This code will print a table of data with these new variables, but it will not save it in your environment!
- ▶ Very common problem that is hard to figure out is you run code that creates new variables, but when you look in your data, you can't find them

summarize and group_by

- ▶ Often we want some summary statistics over groups
 - ▶ In our example, for each year (the group), we want average amount of childcare (the summary statistic)
- ▶ The way this is often done in the tidyverse is to combine `group_by()` with `summarize`
 - ▶ This is similar to the `collapse` command in Stata, but has some significant advantages relative to `collapse`, which we will discuss
- ▶ We will discuss these two functions in turn, before combining them

Use summarize to describe a variable

- ▶ You should think of the summarize command as generating a new data frame with summary statistics
- ▶ You need to specify (1) the name of the variables and (2) the summary statistics

```
summarize(rr, meantotchildcare = mean(childtot))
```

```
##   meantotchildcare  
## 1                NA
```

- ▶ The name here is meantotchildcare and the summary statistic is mean(childtot). In other words, the average value of childtot

Whoops! We need to deal with NAs

```
summarize(rr,  
  meantotchildcare = mean(childdtot, na.rm=T))
```

```
##   meantotchildcare  
## 1              4.687625
```

We can combine and save as an object

```
sumstats <- summarize(rr,  
  meantotchildcare = mean(childdtot, na.rm=T),  
  mediantotchildcare = median(childdtot, na.rm=T))
```

Grouping data

- ▶ Unit of analysis in the data is a particular observation
- ▶ Sometimes these units are organized into groups
- ▶ A lot of times we want to summarize the data by these groups

We might be interested in years and gender, for example

Figure 1. Time Spent on Childcare by Parents, by Educational Attainment, 1965–2008



Source: Authors' estimates based on regression results reported in table 1, columns 1-1 and 1-3.

We can use `group_by` in tidy

```
rr %>% group_by(dataset) %>%  
  summarize(meantotchildcare = mean(childdtot, na.rm=T))
```

- ▶ This will create a tibble that for every unique value of dataset, report the mean of total time spent in childcare in a week

You can group by multiple variables

```
rr %>% group_by(dataset, college) %>%  
  summarize(meantotchildcare = mean(childdtot, na.rm=T))
```

- ▶ Now it groups the data by every unique combination of dataset and college
 - ▶ In other words, for every year, it computes the mean total childcare for individuals who attended college, and those who did not attend college
 - ▶ Equivalent Stata code: collapse (mean) childdtot, by(dataset college)

Reviewing Everything We've Done

- ▶ Load the rug rat race data
- ▶ Select only mothers between 25-34
- ▶ Create a new variable for college prep
- ▶ Summarize that variable over year, and education of mother
- ▶ Do this using the pipes operator

Load data

```
library(tidyverse)
rr <- read_csv("rugratrace.csv")

## Rows: 106020 Columns: 54
## -- Column specification -----
## Delimiter: ","
## dbl (54): state, age, sex, ethnic, under18, under5, ageyngst, student, 
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this
```

Filtering

#Just mothers between 25 and 34

```
mothers2534 <- filter(rr, age<=34, age>=25, mother2==1)
```

Mutate

```
#Create new variable of college prep  
motherscollegeprep <- mutate(mothers2534,  
  childcollegeprep = childtravel + childeduc)
```

Summarize

#Group by year and education

```
motherscollegeprep %>% group_by(dataset, college) %>%  
  summarize(meancollegeprep = mean(childcollegeprep, na.rm=T))
```

Do this all using the pipes operator and save

```
output <- rr %>%  
  filter(age<=34, age>=25, mother2==1) %>%  
  mutate(  
    childcollegeprep = childtravel + childeduc) %>%  
  group_by(dataset, college) %>%  
  summarize(meancollegeprep = mean(childcollegeprep, na.rm=T))
```

```
## `summarise()` has grouped output by 'dataset'. You can override using the  
## `.groups` argument.
```